



Universidad
Indoamérica

UNITÉ 2 · FONDEMENTS DES SYSTÈMES D'EXPLOITATION

Appels système et E/S de bas niveau

Universidad Tecnológica Indoamérica

Laboratoire TD1

Abstraction

- Simplifier et standardiser
- Portabilité du noyau et des programmes (POSIX)
- Faciliter le développement des pilotes
- Stabiliser l'environnement d'exécution

Les abstractions du cours

Fichiers (U3) · processus et mémoire virtuelle (U4) · communication (U5, U6, U8) · threads (U7) · et maintenant les agents (U9)

Protection

- Identification et droits d'utilisation/d'accès
- Isolation entre processus et entre utilisateurs
- Chiffrement des données
- Politiques de sécurité (réseau, fichiers...)

2026: les mêmes principes s'appliquent à un nouvel « utilisateur »: l'**agent d'IA** (identité propre, moindre privilège, audit — U9).

Virtualisation

Portabilité et programmation

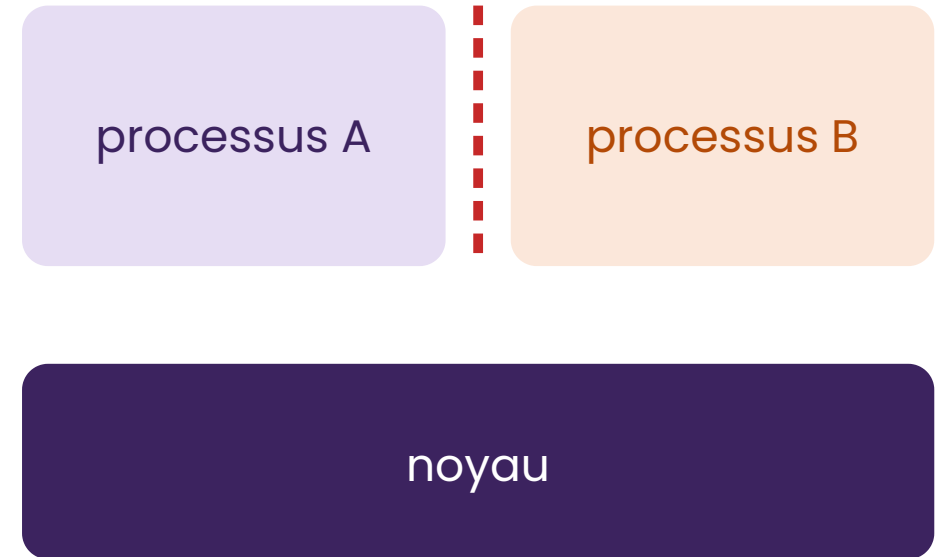
- VFS: une seule interface pour ext4, Btrfs, NFS, /proc ...
- Une vue homogène du matériel (disques USB, SATA et NVMe vus de la même façon)
- POSIX standardise les fonctions du système

Stabilité et évolution

- Isolation des processus: une panne ne fait pas tomber la machine
- Simuler une machine ou un autre OS: QEMU/KVM, WSL 2
- Conteneurs: namespaces + cgroups
- IPv4 sur IPv6

La notion de processus

- Chaque processus a son propre **contexte**
- Une vue **privée** de la mémoire : il ne voit pas celle des autres
- Il est isolé et **se croit seul** : les autres processus et le système lui-même sont cachés



Deux modes d'exécution du processeur

Mode utilisateur (non privilégié)

Le mode par défaut. Le programme ne fait qu'additionner, lire et écrire **dans sa propre mémoire**.

Mode noyau (privilégié)

Pour tout le reste, le processus **demande la permission** avec une instruction spéciale — un appel système — : ouvrir, modifier ou renommer un fichier, allouer de la mémoire, créer ou terminer une tâche...

Le matériel change de mode et l'OS vérifie que l'opération est permise : tout ce qui pourrait nuire à d'autres tâches ou utilisateurs.

Le mode noyau et le noyau

- Certaines instructions (gestion mémoire, interruptions) sont **interdites** en mode utilisateur
- Le noyau **gère** les processus, mais **n'est pas un processus**
- Il gère les ressources et l'accès au matériel; il réside toujours en mémoire; il est chargé au démarrage (U1, A1.3)
- Un processus utilisateur **passe en mode noyau** pour utiliser un service du noyau

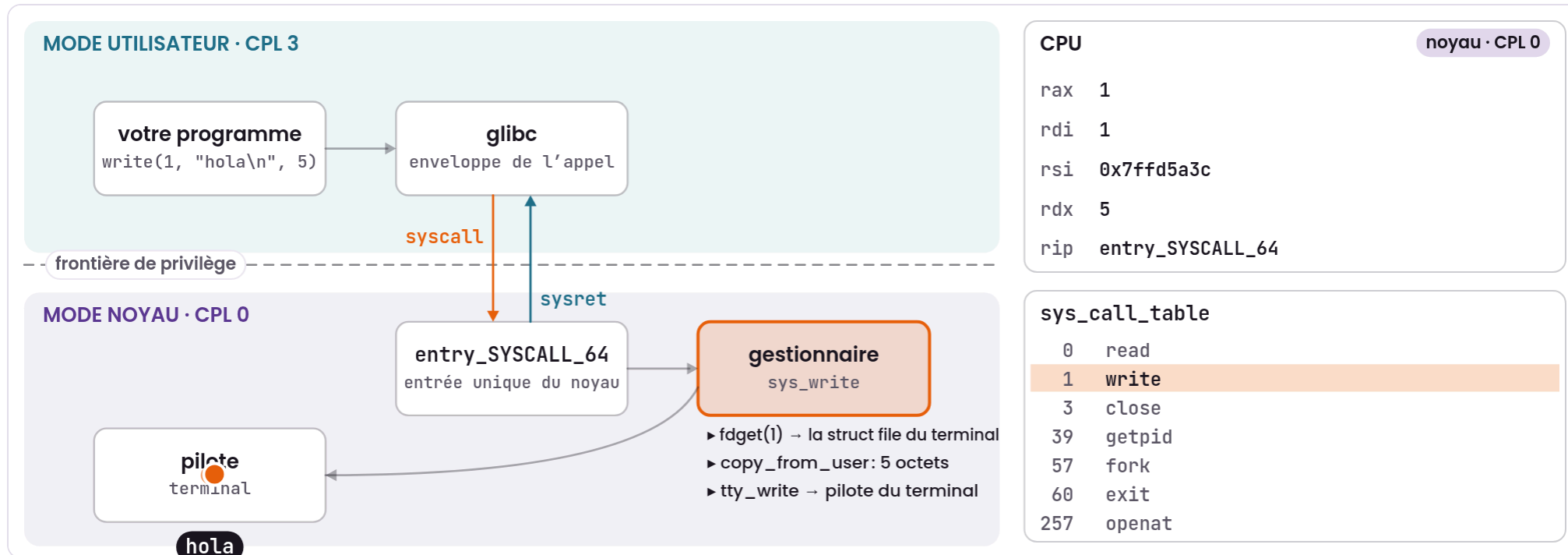
Les appels système: l'interface

- Interface entre le mode utilisateur et le mode noyau
- Le changement de mode est déclenché par une instruction: `ta` (Sparc), `int 0x80` (ancien x86), aujourd'hui `syscall` (x86-64), `svc #0` (ARM64), `ecall` (RISC-V)
- Le noyau a un gestionnaire d'entrée et une table des appels

Les appels système coûtent cher !

L'original disait « plusieurs centaines de cycles ». Mesuré sur la machine du cours: $\approx 0,7 \mu\text{s}$ par itération `read` + `write` (des milliers de cycles).

Le voyage de `write(1, "ho!a\n", 5)`



L'instruction, vue avec objdump

REGISTRE	WRITE(1, BUF, 5)
rax	1 (numéro de write)
rdi	1 (descripteur)
rsi	adresse de buf
rdx	5

Sortie réelle

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-  
gnu/libc.so.6 | grep -A6 -E '^[0-9a-f]+'  
<__write_nocancel@@'  
0000000000129060 <__write_nocancel@@GLIBC_PRIVATE>:  
129060: endbr64  
129064: mov    $0x1,%eax  
129069: syscall  
12906b: cmp    $0xffffffffffffffff, %rax  
129071: ja     129078  
<__write_nocancel@@GLIBC_PRIVATE+0x18>  
129073: ret
```

Erreurs: -1, errno et perror

- Un appel qui échoue renvoie **-1** et laisse le code dans **errno**
- Message: `void perror(const char *s);` ou **%m** dans `fprintf` (extension de la glibc)
- Dans le cours: `exit_if(rc == -1, "write")`

Sortie réelle

```
$ ./echo blabla < . 1>&0  
write: Bad file descriptor  
[exit status 1]
```

C

```
ssize_t rc = write(STDOUT_FILENO, buf, n);  
if (rc == -1)  
    fprintf(stderr, "write: %m\n");  
exit_if(rc == -1, "write");
```

Tracer les appels: **strace**

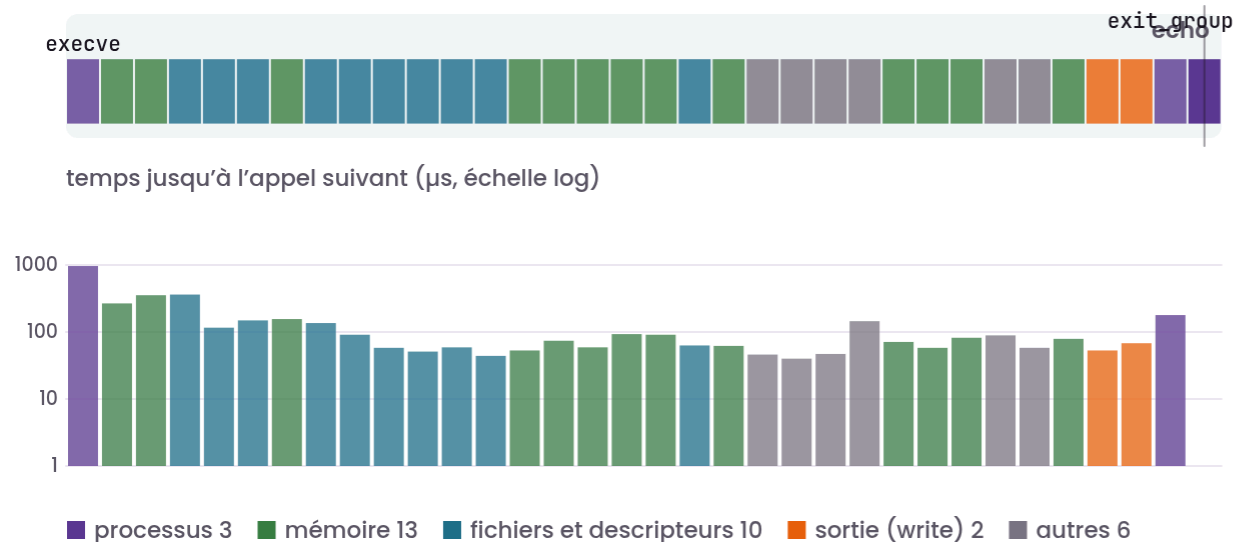
Sortie réelle

```
$ strace -r ./echo hola > /dev/null      # the whole life of the process
0.000000 execve("./echo", [ "./echo", "hola"], 0x7ffff5a76120 /* 129 vars */) = 0
0.000965 brk(NULL)                      = 0x615cdaf80000
0.000267 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x747beb566000
0.000354 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
0.000362 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
0.000116 fstat(3, {st_mode=S_IFREG|0644, st_size=108847, ...}) = 0
0.000149 mmap(NULL, 108847, PROT_READ, MAP_PRIVATE, 3, 0) = 0x747beb54b000
0.000156 close(3)                       = 0
...
```

Sortie réelle

```
0.000089 getrandom("\x70\x5a\xe6\xac\xf7\xee\x72\x1b", 8, GRND_NONBLOCK) = 8
0.000058 munmap(0x747beb54b000, 108847) = 0
0.000079 write(1, "hola", 4)              = 4
0.000053 write(1, "\n", 1)                = 1
0.000068 exit_group(0)                  = ?
0.000179 +++ exited with 0 +++
```

Toute la trace, appel par appel



```

set_tid_address(0x747beb548d68) = 809641
set_robust_list(0x747beb548a20, 24) = 0
rseq(0x747beb548680, 0x21, 0, 0x53053053) = 0
mprotect(0x747beb40e000, 16384, PROT_READ) = 0
mprotect(0x615caa96a000, 4096, PROT_READ) = 0
mprotect(0x747beb5ab000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, ...
getrandom("\x70\x5a\xe6\xac\xf7\xee\x72\x1b", 8, GRND_...
munmap(0x747beb54b000, 108847) = 0
write(1, "hola", 4)      = 4
write(1, "\n", 1)       = 1
exit_group(0)           = ?
+++ exited with 0 +++

```

Appels système et fonctions de bibliothèque

Appels système (man 2)

Inodes: `stat`, `unlink`, `chmod`, `link` ...

Descripteurs: `open`, `read`, `write`, `lseek`,
`close`, `fcntl` ...

Bibliothèque C (man 3)

`fopen`, `fread`, `fprintf`, `fclose` ... **ne sont pas des appels système** : ils utilisent un `FILE *` (avec tampon) au lieu d'un descripteur. `fileno()` convertit l'un en l'autre.

Lire des données: `read`

C

```
ssize_t read(int fd, void *buf, size_t count);
```

- Lit **jusqu'à** `count` octets du fichier de `fd` dans `buf`, qui doit déjà être alloué (`count` $\leq$ sa taille)
- Renvoie le nombre d'octets lus, **0 en fin de fichier** ou `-1` en cas d'erreur
- **Il peut renvoyer moins** d'octets que demandé: tube, terminal, réseau ou signal (`EINTR` s'il n'a rien lu)

Écrire des données: `write`

C

```
ssize_t write(int fd, const void *buffer, size_t count);
```

- Écrit les `count` premiers octets de `buffer` dans le fichier de `fd`
- Renvoie `-1` en cas d'erreur ou le nombre d'octets écrits

Correction de l'original: « s'il écrit moins que `count`, c'est une erreur » n'est pas toujours vrai. Avec des tubes ou des sockets pleins, ou un signal en cours de route, l'écriture peut être **partielle et valide** : recommencez avec le reste (`write_all`).

Où est le bogue?

C

```
/* slide 51 of the original */
while ((byte = read(fd, buf, length))  $\neq$  0) {
    if (byte == -1) {
        perror("read() failed"); exit(1);
    }
    length = length - byte;
    buf = buf + length;      /* BUG */
}
```

C

```
/* fixed: advance by what was READ */
while (length > 0) {
    byte = read(fd, buf, length);
    if (byte == -1 && errno == EINTR)
        continue;
    exit_if(byte == -1, "read");
    if (byte == 0)           /* end of file */
        break;
    length -= byte;
    buf += byte;
}
```

La même boucle avec un fichier et avec un tube

Original (diapositive 51)

```
while ((byte = read(fd, buf, length)) != 0) {
    if (byte == -1) {
        perror("read() failed"); exit(1);
    }
    length = length - byte;
    buf = buf + length;
}
```

buf = 43 length = 0

\$ strace -e trace=read ./prog (pipe)

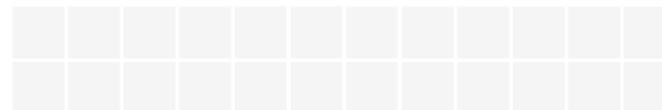
read(3, buf+36, 16) = 9

read(3, buf+43, 7) = 7

read(3, buf+43, 0) = 0

tube : l'écrivain fait 4 write() de 4 + 4 + 9 + 7 octets

tube



mémoire du processus : char buf[24] et ce qui suit

+0 r e a d

+12 () m

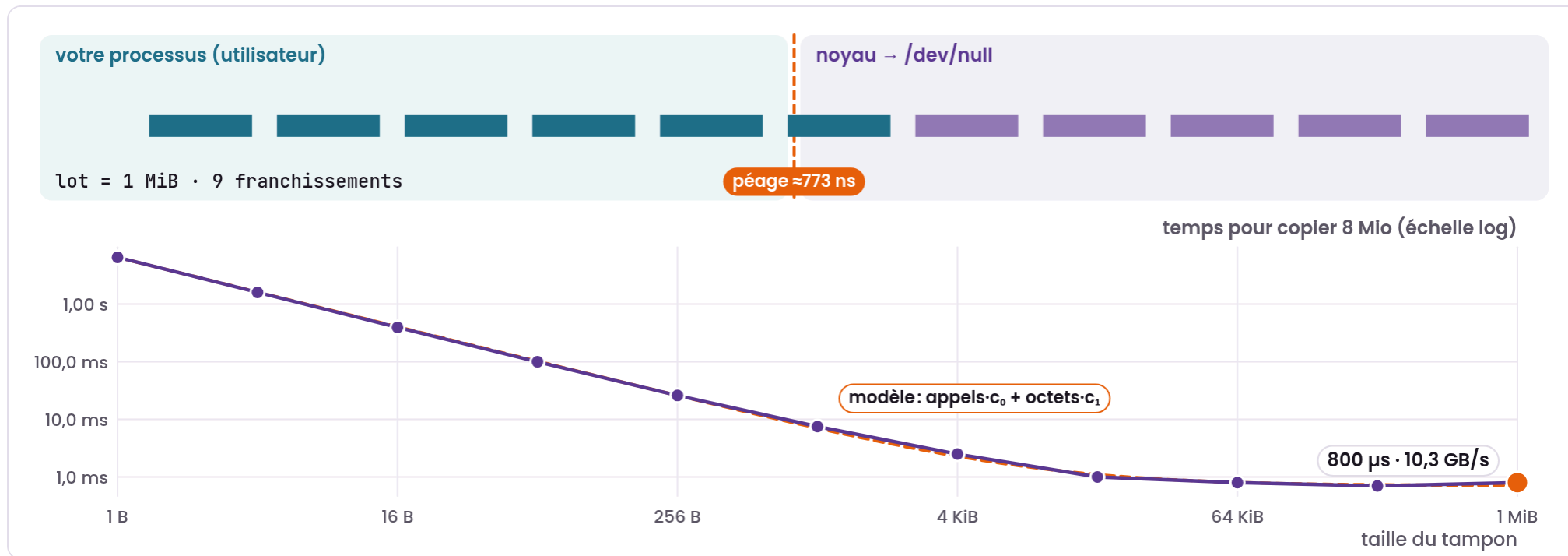
fin de buf[24] · mémoire voisine: autres variables, adresse de retour...

+24 ? ? ? ? ? ? ? ? ? ? ? ?

+36 a y r e t u l e s s

buf = 43

Combien coûte chaque franchissement? Mesuré



Moins de franchissements, même travail

vDSO

`clock_gettime` sans entrer dans le noyau.

io_uring

Files partagées: des centaines d'opérations par appel.

eBPF

Observer les appels dans tout le système sans arrêter les processus.

TD1 corrigé + défi IA

- Compilez et exécutez les 7 exercices; cassez-en un exprès
- Expliquez chaque ligne de `strace ./echo`
- Défi IA: mesurez le coût des appels sur votre machine

Bash

```
make -C tds && tds/tests/run.sh td1
```



Universidad
Indoamérica

À RETENIR

Trois questions

1. Pourquoi le noyau n'est-il pas un processus, et pourquoi n'est-il pas root ?
2. Que renvoie `read` en fin de fichier et que fait la glibc de -9 ?
3. Qu'est-ce qui n'allait pas dans l'exemple de la diapositive 51 ?